Claude AI Mastery

From Beginner to Enterprise AI Engineer

A 100-module professional training program

100 modules • 20 weeks • 6 phases • 5 capstones

AJSSN — Senior AI Architect Training Program

1.0 (April 2026)

The promise

What you will be able to do in 20 weeks

- Design, build, and deploy production-grade Claude applications.
- Architect enterprise RAG, agent, and automation systems.
- Integrate Claude with Slack, Notion, GitHub, Zapier, LangChain, LlamaIndex, databases, and CRMs.
- Ship a portfolio of five capstone projects hosted on GitHub.
- Pass technical interviews for AI Engineer, LLM Engineer, and AI Architect roles.

Course map

Six phases, one hundred modules

Phase	Title	Modules	One-line summary
1	**AI & Claude Foundations**	Modules 1–15	Build a correct mental model of AI, LLMs, and Claude. Learn to prompt, evaluate, and reason about limitations before writing a single line of code.
2	**Claude Productivity & Practical Use**	Modules 16–35	Apply Claude to daily work: documentation, coding, research, meetings, email, automation. Convert prompt skills into productivity leverage.
3	**Claude API & Development**	Modules 36–55	Move from chat to code. Call the Claude API from Python and Node, manage keys, stream output, and ship your first real AI app.
4	**Advanced AI Engineering**	Modules 56–80	Graduate to RAG, embeddings, vector databases, agents, tools, and framework integrations. Build systems that reason over private data.
5	**Enterprise Claude AI Systems**	Modules 81–95	Operate Claude at scale: architecture, security, cost, observability, CI/CD, and governance.
6	**Real-World AI Projects**	Modules 96–100	Ship five portfolio capstones with architecture, code, and GitHub templates.

Connectors catalog

Where Claude plugs into the rest of the stack

Connector	Primary use	Surface
Slack	Deliver Claude answers to channels, DMs, and slash commands.	`Slack app + Events API + Claude API`
Notion	Index pages into RAG, auto-generate docs back into Notion.	`Notion API + Claude`
Zapier	No-code triggers (email, forms, CRM) calling Claude.	`Zapier Anthropic action`
GitHub	PR reviews, test generation, release notes, issue triage.	`GitHub Actions + Octokit + Claude`
LangChain	Chains, memory, agents, retrievers.	`langchain-anthropic`
LlamaIndex	Document indexing, query engines, sub-question routing.	`llama-index-llms-anthropic`
Google Workspace	Drive/Docs/Sheets automation, Gmail drafting.	`Google APIs + Claude`
Databases	Text-to-SQL, schema explanation, data QA.	`psycopg2/SQLAlchemy + Claude`
CRM (Salesforce/HubSpot)	Call summaries, lead enrichment, next-best-action.	`Salesforce/HubSpot API + Claude`

Instructor plan at a glance

20-week pacing, one lab per evening (optional)

Pace: 5 modules per week, 1 lab per evening (optional), 1 capstone across 2 weeks.
Cohort size: 8–25 students recommended.
Delivery: In-person bootcamp (full-day) • Part-time evening cohort (3h x 2/week) • Self-paced with weekly office hours

Wk	Ph	Focus
1	1	AI, LLMs, Claude; first prompts
2	1	Prompt engineering, structure, CoT, personas, few-shot
3	1	Sampling, long context, hallucinations, ethics, P1 capstone
4	2	Documentation, code gen, review, debugging, tests
5	2	Research, data, SQL, meetings, email
6	2	Slides, translation, editing, creative, optimization
7	2	Templating, workflows, artifacts, files, P2 capstone
8	3	API, auth, Python, TS, streaming
9	3	Messages, tools, multi-tool, vision, PDFs
10	3	Limits, errors, async, FastAPI bot, deploy

Wk	Ph	Focus
11	3	Slack, Discord, CLI, SMS, P3 capstone
12	4	RAG, embeddings, vector DBs, Pinecone
13	4	Weaviate, Chroma, pgvector, chunking, hybrid
14	4	Reranking, RAG quality, eval, agents, memory
15	4	Tools at scale, guardrails, LangChain, LCEL, LangGraph
16	4	LlamaIndex, production RAG, Zapier/n8n/Airflow, P4 capstone
17	5	Enterprise arch, microservices, multi-tenant, workflows, security
18	5	Injection, PII, audit, scaling, cost
19	5	Monitoring, CI/CD, canary, upgrades, P5 capstone
20	6	Capstone projects + demos

Phase 1

AI & Claude Foundations

Modules 1–15

Build a correct mental model of AI, LLMs, and Claude. Learn to prompt, evaluate, and reason about limitations before writing a single line of code.

Phase 1 — Goals

AI & Claude Foundations

Summary

Build a correct mental model of AI, LLMs, and Claude. Learn to prompt, evaluate, and reason about limitations before writing a single line of code.

Learning outcomes

- Explain what an LLM is and isn't.
- Use the Claude chat interface proficiently.
- Write structured prompts that reliably produce quality output.
- Identify AI risks, hallucinations, and ethical boundaries.

Phase 1 — Slide plan (1 of 3)

AI & Claude Foundations

- Course overview and promise
- Why this course exists in 2026
- The AI stack in one picture
- LLM essentials — tokens, embeddings, attention
- Meet the Claude family (Opus / Sonnet / Haiku)
- The Claude interface: Projects, Artifacts, Files

Phase 1 — Slide plan (2 of 3)

AI & Claude Foundations

- Capabilities and limits — one honest slide each
- Prompt engineering: the 6-part template
- Structured prompting with XML tags and JSON schemas
- Chain of thought and self-critique
- Personas, style control, few-shot
- Temperature and determinism

Phase 1 — Slide plan (3 of 3)

AI & Claude Foundations

- Long context strategies
- Hallucinations and groundedness
- Ethics and safety mindset
- Phase 1 capstone brief

Module 1 — Introduction to Artificial Intelligence

Phase 1: AI & Claude Foundations

- What — AI is the engineering discipline of making machines perform tasks that normally require human intelligence — perception, reasoning, language, and decision-making.
- Why — Every major industry is being re-tooled around AI. Understanding the field gives you a map of where LLMs, agents, and Claude fit.
- When — Use AI when the problem is pattern-rich, data-rich, and rules-poor — or when scale makes human-only workflows uneconomical.
- Simple — AI = software that learns patterns from examples instead of being told every rule.
- Use cases — IT: AI-assisted ticket triage; DevOps: anomaly detection in logs; Software: code completion
- Connectors — Slack, Notion, GitHub

Module 2 — Introduction to Large Language Models (LLMs)

Phase 1: AI & Claude Foundations

- What — LLMs are transformer-based neural networks trained to predict the next token over enormous text corpora, which gives them surprisingly general language and reasoning abilities.
- Why — LLMs are the substrate under Claude, ChatGPT, Gemini, and nearly every modern AI product. Everything else in this course sits on top of this idea.
- When — When input and output are naturally text (or can be made text: JSON, code, tables, transcripts).
- Simple — An LLM is autocomplete on steroids — trained on so much text it learned to reason while predicting the next word.
- Use cases — Summarize tickets; Draft release notes; Extract entities from PDFs
- Connectors — GitHub, Slack

Module 3 — Claude AI Overview and Model Family

Phase 1: AI & Claude Foundations

- What — Claude is Anthropic's family of frontier LLMs (Opus, Sonnet, Haiku) designed for safety, steerability, and enterprise use.
- Why — Choosing the right Claude tier is the single biggest lever on cost, latency, and quality in any production system.
- When — Opus for hardest reasoning; Sonnet for the default production workhorse; Haiku for high-volume, latency-sensitive flows.
- Simple — Opus = brain surgeon. Sonnet = senior engineer. Haiku = fast intern.
- Use cases — Tiered routing by query difficulty; Draft/refine pattern (Haiku drafts, Sonnet refines); Batch classification on Haiku
- Connectors — Slack, Google Workspace

Phase 2

Claude Productivity & Practical Use

Modules 16–35

Apply Claude to daily work: documentation, coding, research, meetings, email, automation. Convert prompt skills into productivity leverage.

Phase 2 — Goals

Claude Productivity & Practical Use

Summary

Apply Claude to daily work: documentation, coding, research, meetings, email, automation. Convert prompt skills into productivity leverage.

Learning outcomes

- Automate 30%+ of repetitive knowledge work with Claude.
- Build reusable prompt templates and Projects.
- Design multi-step Claude workflows with Artifacts and Files.

Phase 2 — Slide plan (1 of 4)

Claude Productivity & Practical Use

- Phase 2 goals
- Claude for technical documentation
- Claude for code generation
- Claude for code review
- Claude for debugging
- Claude for automated testing

Phase 2 — Slide plan (2 of 4)

Claude Productivity & Practical Use

- Claude for research & literature review
- Claude for data analysis
- Claude for SQL
- Claude for meeting notes
- Claude for email and comms
- Claude for slides and decks

Phase 2 — Slide plan (3 of 4)

Claude Productivity & Practical Use

- Translation and localization
- Editing and tone control
- Creative ideation
- Prompt optimization (promptfoo, DSPy)
- Prompts as versioned templates
- Multi-step workflows

Phase 2 — Slide plan (4 of 4)

Claude Productivity & Practical Use

- Artifacts and Files
- Phase 2 capstone brief

Module 16 — Claude for Technical Documentation

Phase 2: Claude Productivity & Practical Use

- What — Use Claude to draft, maintain, and refactor technical docs — READMEs, API references, runbooks, architecture notes.
- Why — Documentation is chronically under-resourced; Claude collapses a 1-day task into 30 minutes while improving consistency.
- When — Start drafting from code, diff summaries, or meeting transcripts; update when APIs change.
- Simple — Hand Claude the code and say 'write the docs.'
- Use cases — Generate README from repo; Runbook from incident postmortem; Changelog from git log
- Connectors — GitHub, Notion

Module 17 — Claude for Code Generation

Phase 2: Claude Productivity & Practical Use

- What — Generate functions, classes, scaffolding, and whole modules from intent, tests, or API specs.
- Why — Most code is glue. Claude removes the glue tax and lets engineers focus on design and tests.
- When — Greenfield modules, boilerplate, migrations, and repetitive patterns.
- Simple — Describe what you want; Claude writes the first draft.
- Use cases — Boilerplate endpoints; Terraform modules; Data-loading scripts
- Connectors — GitHub, Databases

Module 18 — Claude for Code Review and Refactoring

Phase 2: Claude Productivity & Practical Use

- What — Use Claude to review diffs, catch bugs, suggest refactors, and enforce style — on PRs or locally.
- Why — A tireless senior reviewer cuts merge cycle time and improves quality across the team.
- When — Before merging, on large refactors, or when onboarding new engineers.
- Simple — Paste the diff, ask 'what would a senior reviewer say?'
- Use cases — Auto-comment on PRs; Security-focused review of IaC; Legacy refactor assistant
- Connectors — GitHub, Slack

Phase 3

Claude API & Development

Modules 36–55

Move from chat to code. Call the Claude API from Python and Node, manage keys, stream output, and ship your first real AI app.

Phase 3 — Goals

Claude API & Development

Summary

Move from chat to code. Call the Claude API from Python and Node, manage keys, stream output, and ship your first real AI app.

Learning outcomes

- Call Claude from Python and Node with auth, retries, and streaming.
- Build chatbots, summarizers, classifiers, and extractors.
- Deploy a Claude app to a cloud container.

Phase 3 — Slide plan (1 of 3)

Claude API & Development

- From chat to code
- The Messages API
- Auth and key management
- First Python call
- First TypeScript call
- Streaming responses

Phase 3 — Slide plan (2 of 3)

Claude API & Development

- System prompts and multi-turn
- Tool use basics
- Multi-tool orchestration
- Vision and PDFs
- Rate limits and retries
- Observability

Phase 3 — Slide plan (3 of 3)

Claude API & Development

- Async and batch
- FastAPI chatbot
- Dockerize and deploy
- Slack / Discord / CLI / SMS integrations
- Phase 3 capstone brief

Module 36 — Introduction to the Claude API

Phase 3: Claude API & Development

- What — The Claude REST API exposes Claude as a JSON service with messages, tools, streaming, vision, and files.
- Why — Moves Claude from a chat UI into your products, scripts, and pipelines.
- When — When a feature needs to be programmatic, repeatable, or embedded.
- Simple — You POST a conversation; Claude POSTs back a reply.
- Use cases — Replace rules engines with LLM classification; Dynamic text generation in apps; Pipeline narration and summarization
- Connectors — GitHub, Databases

Module 37 — API Authentication and Key Management

Phase 3: Claude API & Development

- What — API keys are the credential that authorizes your calls — treat them like database passwords.
- Why — Leaked keys cost money and can violate data protection commitments.
- When — From the first call onward.
- Simple — Never commit keys; use env vars and secret managers.
- Use cases — Per-team API keys; Rotated keys in CI; Break-glass keys for incidents
- Connectors — GitHub Actions, AWS/GCP

Module 38 — First Real API Call — Python

Phase 3: Claude API & Development

- What — Walk through a complete Python example: env setup, client, messages, error handling, logging.
- Why — Get from 'works on my machine' to a minimally robust call.
- When — Your very first integration.
- Simple — Put it behind a function with retries and logs.
- Use cases — Utility function in any Python app; Jupyter analysis; Microservice handler
- Connectors — GitHub

Phase 4

Advanced AI Engineering

Modules 56–80

Graduate to RAG, embeddings, vector databases, agents, tools, and framework integrations. Build systems that reason over private data.

Phase 4 — Goals

Advanced AI Engineering

Summary

Graduate to RAG, embeddings, vector databases, agents, tools, and framework integrations. Build systems that reason over private data.

Learning outcomes

- Design and implement end-to-end RAG pipelines.
- Build Claude agents with tool-use, memory, and guardrails.
- Integrate LangChain and LlamaIndex where they add real value.

Phase 4 — Slide plan (1 of 4)

Advanced AI Engineering

- Phase 4 goals
- Why RAG
- Embeddings and geometry
- Embedding providers compared
- Vector databases overview
- Pinecone / Weaviate / Chroma / pgvector

Phase 4 — Slide plan (2 of 4)

Advanced AI Engineering

- Chunking strategies
- Hybrid search with RRF
- Reranking: the multiplier
- RAG quality metrics
- Evaluating pipelines
- Agents — ReAct, Plan-Execute, Reflexion

Phase 4 — Slide plan (3 of 4)

Advanced AI Engineering

- Agent memory patterns
- Tools at scale with permissions
- Guardrails and safety
- LangChain and LCEL
- LangGraph stateful agents
- LlamaIndex indexing and query engines

Phase 4 — Slide plan (4 of 4)

Advanced AI Engineering

- End-to-end production RAG
- Automation pipelines
- Phase 4 capstone brief

Module 56 — Retrieval-Augmented Generation (RAG) — Foundations

Phase 4: Advanced AI Engineering

- What — RAG augments an LLM at query time with relevant snippets retrieved from a knowledge base, so answers are grounded in your data.
- Why — Claude can now answer about proprietary content, stay current, and cite sources.
- When — Whenever the needed knowledge isn't in the model's weights.
- Simple — Search first; ask Claude second.
- Use cases — Internal wiki Q&A; Product support; Compliance lookup
- Connectors — Notion, Google Workspace, Databases

Module 57 — Embeddings — Concepts and Geometry

Phase 4: Advanced AI Engineering

- What — Embeddings map text to dense vectors whose distance approximates semantic similarity.
- Why — They enable semantic search, clustering, and recommendation over unstructured data.
- When — Whenever you need 'similar' over text.
- Simple — Embeddings turn meaning into math.
- Use cases — Similar-ticket clustering; Deduplication; Search ranking features
- Connectors — Databases (pgvector)

Module 58 — Embedding Providers — Hosted and Local

Phase 4: Advanced AI Engineering

- What — Survey of hosted (Voyage, OpenAI, Cohere) and local (BGE, E5, Instructor) embedding models — trade-offs.
- Why — Model choice sets the quality ceiling for RAG.
- When — Before committing to an index.
- Simple — Host for scale; local for control and cost.
- Use cases — Hosted for fast start; Local for air-gapped deployments; Domain-tuned for highest quality
- Connectors — Databases

Phase 5

Enterprise Claude AI Systems

Modules 81–95

Operate Claude at scale: architecture, security, cost, observability, CI/CD, and governance.

Phase 5 — Goals

Enterprise Claude AI Systems

Summary

Operate Claude at scale: architecture, security, cost, observability, CI/CD, and governance.

Learning outcomes

- Draft an enterprise AI reference architecture.
- Implement security, PII redaction, and audit logging.
- Set up monitoring, cost guardrails, and CI/CD for AI.

Phase 5 — Slide plan (1 of 3)

Enterprise Claude AI Systems

- The enterprise reference architecture
- AI microservices and contracts
- Multi-tenant design
- Durable workflows (Temporal)
- Security: secrets, network, identity
- Prompt injection and robustness

Phase 5 — Slide plan (2 of 3)

Enterprise Claude AI Systems

- PII, GDPR, HIPAA
- Audit logging and compliance
- Scaling: caching and queuing
- Cost optimization and routing
- Monitoring and SLOs
- CI/CD for AI

Phase 5 — Slide plan (3 of 3)

Enterprise Claude AI Systems

- Canary and online eval
- Model upgrade playbook
- Phase 5 capstone brief

Module 81 — Enterprise AI Reference Architecture

Phase 5: Enterprise Claude AI Systems

- What — A layered architecture: ingress, orchestration, model gateway, RAG/tool layer, safety, observability, and governance.
- Why — Without a reference, teams reinvent wheels and create inconsistent security and cost profiles.
- When — Before the second team ships a Claude feature.
- Simple — A published blueprint every team builds against.
- Use cases — Shared platform for 5+ AI products; Reg-heavy industries; Global enterprises
- Connectors — Databases, Slack, GitHub

Module 82 — AI Microservices Design

Phase 5: Enterprise Claude AI Systems

- What — Break Claude features into composable microservices (embeddings, retrieval, orchestration, models) with clear contracts.
- Why — Scales the organization: teams move independently.
- When — Past 5 features or 3 teams.
- Simple — Small services, strong contracts.
- Use cases — Shared rerank service; Shared embeddings service; Shared policy service
- Connectors — Databases

Module 83 — Multi-Tenant Claude Systems

Phase 5: Enterprise Claude AI Systems

- What — Design for many customers/users with isolation, quotas, and customization.
- Why — SaaS realities: noisy neighbors, data isolation, per-tenant prompts.
- When — Second paying customer.
- Simple — Tenant id everywhere — logs, vectors, tools, limits.
- Use cases — Support SaaS; Legal SaaS; Analytics SaaS
- Connectors — Databases

Phase 6

Real-World AI Projects

Modules 96–100

Ship five portfolio capstones with architecture, code, and GitHub templates.

Phase 6 — Goals

Real-World AI Projects

Summary

Ship five portfolio capstones with architecture, code, and GitHub templates.

Learning outcomes

- Deliver five end-to-end capstone projects.
- Publish portfolio-ready GitHub repositories.
- Present each project with architecture and ROI narrative.

Phase 6 — Slide plan (1 of 3)

Real-World AI Projects

- Capstone 1 — IT Helpdesk Assistant: architecture
- Capstone 1 — build steps and demo
- Capstone 2 — Enterprise Knowledge Chatbot: architecture
- Capstone 2 — build steps and demo
- Capstone 3 — AI Documentation Generator: architecture
- Capstone 3 — build steps and demo

Phase 6 — Slide plan (2 of 3)

Real-World AI Projects

- Capstone 4 — DevOps Automation Bot: architecture
- Capstone 4 — build steps and demo
- Capstone 5 — AI Research Assistant: architecture
- Capstone 5 — build steps and demo
- Demo day rubric and evaluation
- Career path — AI Engineer, LLM Engineer, AI Architect

Phase 6 — Slide plan (3 of 3)

Real-World AI Projects

- What to do next

Module 96 — Capstone 1 — Claude AI IT Helpdesk Assistant

Phase 6: Real-World AI Projects

- What — An agent that triages, classifies, answers, and (with approval) resolves IT tickets end-to-end.
- Why — Helpdesk is the highest-ROI place to deploy AI in most enterprises.
- When — Now — before your backlog grows.
- Simple — A tireless Tier-1 analyst backed by your knowledge base.
- Use cases — Password reset; Software access requests; How-to questions
- Connectors — Slack, Databases, CRM/ITSM

Module 97 — Capstone 2 — Enterprise Knowledge AI Chatbot (RAG)

Phase 6: Real-World AI Projects

- What — Company-wide chatbot that answers from Notion, Google Drive, and GitHub with citations and ACLs.
- Why — Kills the 'where is this document?' problem; dramatically improves onboarding.
- When — Once you have >50 active employees and a content sprawl.
- Simple — A single search box that actually finds and answers things.
- Use cases — New-hire onboarding; Policy Q&A; Customer-facing self-serve docs
- Connectors — Notion, Google Workspace, GitHub, Slack

Module 98 — Capstone 3 — AI Documentation Generator

Phase 6: Real-World AI Projects

- What — A tool that reads code + diffs + tests and produces docs, READMEs, and changelogs — continuously.
- Why — Docs are never done. This keeps them current with minimal human cost.
- When — Once you have 3+ services or 1 public API.
- Simple — Merge a PR → docs get updated automatically.
- Use cases — API reference updates; Changelog generation; Onboarding docs
- Connectors — GitHub, Notion

Module 99 — Capstone 4 — DevOps AI Automation Bot

Phase 6: Real-World AI Projects

- What — A bot that triages alerts, drafts runbook steps, proposes fixes, and (with approval) executes safe commands.
- Why — Shrinks MTTR and frees on-call engineers from repetitive toil.
- When — When you have enough alert volume to justify automation.
- Simple — Alerts come in → bot summarizes, diagnoses, and (sometimes) fixes.
- Use cases — CrashLoopBackOff triage; Elevated-latency investigations; Deployment rollback suggestion
- Connectors — Slack, Databases, GitHub

Module 100 — Capstone 5 — AI Research Assistant

Phase 6: Real-World AI Projects

- What — A multi-agent research assistant that searches, reads, synthesizes, and cites across web and private sources.
- Why — Demonstrates agents, RAG, and tool use at their limits; produces portfolio-grade deliverables.
- When — Capstone of capstones.
- Simple — Tell it a topic; get a cited report.
- Use cases — Competitor analysis; Policy memo; Market entry report
- Connectors — Notion, Google Workspace, Databases

Capstone Portfolio

Five shippable, portfolio-ready projects

Capstone 1 — Claude AI IT Helpdesk Assistant

Capstone 1 — architecture

Problem

Helpdesk is the highest-ROI place to deploy AI in most enterprises.

Stack

Stack: FastAPI + Slack + pgvector (RAG) + tool-use (ticket system, password reset, MFA) + guardrails + approval queue.

Flow

```
Slack -> Agent -> RAG (KB) -> Tools (tickets/reset) -> Approval (if risky) -> Slack reply -> Audit
```

Capstone 1 — Claude AI IT Helpdesk Assistant

Capstone 1 — build steps & demo

- 1. Provision Postgres+pgvector and Redis.
- 2. Create Slack app; wire Events API to /slack/events.
- 3. Ingest a KB snapshot into pgvector with source URLs.
- 4. Implement tool functions with dry-run flags.
- 5. Add guardrails (prompt injection, PII, forbidden topics).
- 6. Wire approval queue (Slack interactive message).
- 7. Deploy to Cloud Run; add dashboards.
- 8. Pilot with IT team for 2 weeks; review every trace.
- 9. Roll out to org; keep human-in-loop for destructive actions.

Labs Beginner: Implement 'search_kb' and 'create_ticket' tools with mocks. Intermediate: Add an approval step for destructive tools. Advanced: Measure deflection rate and MTTR before/after launch.

Capstone 2 — Enterprise Knowledge AI Chatbot (RAG)

Capstone 2 — architecture

Problem

Kills the 'where is this document?' problem; dramatically improves onboarding.

Stack

Stack: FastAPI + pgvector (hybrid) + Cohere/bge rerank + Claude Sonnet + Slack + web UI (Next.js). Per-tenant ACLs enforced at retrieval time.

Flow

```
Connectors -> Chunk -> Embed -> pgvector
User -> UI -> API -> Hybrid retrieve -> Rerank -> Claude -> Answer + [citations] -> Feedback
```

Capstone 2 — Enterprise Knowledge AI Chatbot (RAG)

Capstone 2 — build steps & demo

- 1. Stand up Postgres+pgvector and connector keys.
- 2. Build ingest jobs per source with incremental updates.
- 3. Chunk (recursive, 800/120) + embed (bge-base or Voyage).
- 4. Serve hybrid retrieve + reranker + Claude + cite.
- 5. Ship Slack + web UI with feedback capture.
- 6. Enforce ACLs at retrieval.
- 7. Build eval set from real questions; run RAGAS nightly.
- 8. Launch to a pilot team; iterate for 2 weeks.
- 9. Expand org-wide with a support rotation.

Labs Beginner: Ingest Notion and serve Slack answers. Intermediate: Add Drive + GitHub + hybrid search + reranker. Advanced: Add ACL filtering + weekly eval report + thumbs feedback loop.

Capstone 3 — AI Documentation Generator

Capstone 3 — architecture

Problem

Docs are never done. This keeps them current with minimal human cost.

Stack

Stack: GitHub Action + Claude + docs repo. Analyzes diff + surrounding files; proposes doc PRs.

Flow

```
PR merged -> Action -> Claude analyzes diff -> Updates /docs -> Opens docs PR -> Human review
```

Capstone 3 — AI Documentation Generator

Capstone 3 — build steps & demo

- 1. Install docs site (Docusaurus/MkDocs).
- 2. Write ai_docs.py: pull PR diff; fetch context; call Claude with style guide.
- 3. Write generated markdown to /docs; open PR via peter-evans action.
- 4. Add Vale linting in CI.
- 5. Add 'skip-docs' label to bypass.
- 6. Track docs drift metric; publish weekly.

Labs Beginner: Generate a README from one repo. Intermediate: Wire a GitHub Action; auto-open docs PRs. Advanced: Add Vale linting + screenshot capture + ADR template detection.

Capstone 4 — DevOps AI Automation Bot

Capstone 4 — architecture

Problem

Shrinks MTTR and frees on-call engineers from repetitive toil.

Stack

Stack: PagerDuty/Opsgenie webhooks → service → Claude + tools (logs, metrics, K8s read-only, runbook KB) → Slack war-room → approval for writes.

Flow

```
Alert -> Bot -> Gather (logs/metrics/k8s) -> Claude RCA -> Suggest action -> Approval (Slack) -> Execute -> Post-incident summary
```

Capstone 4 — DevOps AI Automation Bot

Capstone 4 — build steps & demo

- 1. Wire PagerDuty webhook to /alerts.
- 2. Build read-only tools first (logs, metrics, pod status).
- 3. Claude agent drafts RCA + next steps.
- 4. Post to war-room Slack channel.
- 5. Add gated write tools with Slack approval UI.
- 6. Record every run for post-incident review.
- 7. Expand tool inventory gradually with ownership approvals.

Labs Beginner: Read-only bot that fetches pod status and summarizes. Intermediate: Add metrics + logs queries + RCA summary. Advanced: Add gated write actions (restart deploy) with dual approval.

Capstone 5 — AI Research Assistant

Capstone 5 — architecture

Problem

Demonstrates agents, RAG, and tool use at their limits; produces portfolio-grade deliverables.

Stack

Stack: LangGraph plan-execute + web search tool + PDF tool + private RAG + Claude Opus planner / Sonnet executor. Output: markdown + docx + slide deck.

Flow

```
Topic -> Plan (Opus) -> Search (web+private) -> Read -> Synthesize (Sonnet) -> Cite -> Export (md/docx/pptx)
```

Capstone 5 — AI Research Assistant

Capstone 5 — build steps & demo

- 1. Define the graph (plan, search, read, synth, cite).
- 2. Wire web search + private RAG tools.
- 3. Opus for planning; Sonnet for per-section writing.
- 4. Add a 'verify citations' node that rejects unsupported claims.
- 5. Export to markdown + docx + pptx.
- 6. Evaluate with expert reviewers on 5 topics; iterate.

Labs Beginner: Produce a 3-page research memo with 10 citations. Intermediate: Add private RAG + parallel reading of 20 docs. Advanced: Produce a full report + companion slide deck + exec summary.

Live Demos

Five rehearsed instructor scripts

Demo 1 — From claude.ai to API in 15 minutes

Demo 1 — instructor beats

Scene

Open claude.ai, send a prompt, save the same prompt as a Python function.

Beats

- Open claude.ai; run 'Summarize this policy in 5 bullets'.
- Open VS Code; create policy_summarizer.py with the same prompt.
- pip install anthropic; set ANTHROPIC_API_KEY.
- Run the script; show identical quality.
- Refactor into a function that accepts a path and returns JSON.
- Add retries and structured logging.
- Close: 'You now have a reusable productivity tool.'

Gotchas Missing env var — fail loudly, then fix. • Windows encoding — set UTF-8.

Demo 2 — Tool use: building a calculator agent

Demo 2 — instructor beats

Scene

Show Claude refusing to do long arithmetic, then solving it with a calculator tool.

Beats

- Ask Claude to compute 4712 * 9183; point out the error.
- Define a calculator tool; re-run; show correct answer.
- Add a 'get_time' tool for today's date; demonstrate freshness.
- Log each tool call and argument.
- Close: 'Tools = superpowers; observability is mandatory.'

Gotchas Tool input schema mismatch — show error, correct. • Infinite tool loops — add step cap.

Demo 3 — RAG in 40 lines of Python

Demo 3 — instructor beats

Scene

Build a tiny RAG over 3 markdown files using Chroma, then extend with citations.

Beats

- Create /docs with 3 policy.md files.
- pip install chromadb; build collection; add chunks.
- Query 'What is our refund window?'; show top-3.
- Call Claude with context and require [id] citations.
- Swap embeddings; show improvement.
- Close: 'RAG is simple — quality is in the details.'

Gotchas Path issues with Chroma persistence — use absolute. • Forgetting to embed; keep embedder consistent for query + doc.

Demo 4 — Deploy a Slack bot in one sitting

Demo 4 — instructor beats

Scene

From zero to a /ask command in Slack, deployed to Cloud Run.

Beats

- Create Slack app; enable Events + slash commands.
- Scaffold FastAPI with /slack/events.
- Acknowledge in <3s; do work async.
- Deploy with 'gcloud run deploy'.
- Configure Slack with the public URL.
- Run /ask in Slack; show Claude answering.
- Close: 'Productionization is now a Wednesday afternoon.'

Gotchas Signature verification — must implement. • Slack 3s timeout — acknowledge first; work later.

Demo 5 — Canary a prompt change with online evaluation

Demo 5 — instructor beats

Scene

Route 10% of traffic to a new prompt; watch dashboards; promote.

Beats

- Show the current prompt and its quality dashboard.
- Introduce v2 prompt; add feature flag.
- Deploy; route 10% to v2.
- Compare CSAT + cost + latency on matched samples.
- Promote if better; rollback if not.
- Close: 'Prompts are code; treat them like code.'

Gotchas Skewed sampling — stratify. • False wins on tiny samples — require minimums.

Weekly assignments

20 deliverables across 20 weeks

Wk	Assignment	Description
1	**Prompt Library v1**	Ship 10 role-based prompts to a GitHub repo with examples and a CHANGELOG.
2	**5-shot Classifier**	Label 50 support tickets; build a 5-shot Claude classifier; achieve >=90% accuracy on a holdout of 10.
3	**Phase 1 Capstone**	Personal Claude Workstation: 3 Projects, 10 templates, 20-prompt eval set, privacy checklist.
4	**README-from-Repo**	Generate a README for an old repo; pass Vale lint with 0 warnings.
5	**CSV to Executive Brief**	Take a messy CSV; produce 5 insights with charts and a 1-page memo.
6	**Prompt A/B**	Run promptfoo on 2 prompts over 30 inputs; publish the report.
7	**Phase 2 Capstone**	Personal Automation Suite: 3 automations saving >=2h/week; README with metrics.
8	**CLI Tool on PyPI**	Publish a pip-installable Claude CLI.
9	**3-Tool Agent**	Agent with search, DB, email; 10 tested scenarios.
10	**Deployed Endpoint**	FastAPI service on Cloud Run; /health, /chat; public URL.
11	**Phase 3 Capstone**	Deployable microservice with /classify, /summarize, /chat; CI + tests.
12	**Mini-RAG**	RAG over 50 markdown files with citations; RAGAS groundedness >=0.8.
13	**Hybrid vs Vector-Only**	Run both; publish accuracy/latency/cost comparison on 50 queries.
14	**Reflexion Agent**	Agent that self-critiques; beat the non-reflective baseline by >=5% on eval.
15	**LangGraph Approval Agent**	Agent with human-in-loop approval node; full trace capture.
16	**Phase 4 Capstone**	Enterprise Knowledge Assistant across 3 sources with ACLs.
17	**Reference Architecture Doc**	Write a 4-page enterprise reference architecture ADR.
18	**Cost & Routing Plan**	Budget per feature, Haiku/Sonnet/Opus routing design, prompt caching plan.
19	**Phase 5 Capstone**	Platform reference repo: gateway, router, RAG, eval, obs, infra.
20	**Capstone Demo**	Ship one of the five capstones; present in demo day.

Practice exercises

Short drills between modules

- Rewrite a vague prompt into 6-section structure (role, context, task, format, rules, example).
- Build a JSON-schema-validated classifier with pydantic retries.
- Chunk a 60-page PDF three ways; compare RAG recall.
- Replace a bash script with a Claude-powered Python script doing the same job with explanations.
- Add citations to an existing RAG pipeline and measure groundedness.
- Port a prompt from Sonnet 4.4 -> 4.6; keep parity.
- Red-team your own Claude app with 20 adversarial prompts.
- Write an incident runbook that an on-call engineer can follow with Claude.
- Publish a 'lessons learned' post from a failed prompt and what fixed it.
- Refactor an agent from ReAct to Plan-Execute; compare cost and success rate.

Capstone rubric

0–5 per criterion (0=missing, 3=meets, 5=exemplary)

Criterion	Description
Problem framing	Clear user, pain, and ROI narrative.
Architecture	Diagrams + ADRs; tradeoffs documented.
Implementation quality	Code structure, tests, docs, CI.
RAG / Tool integration	Correct retrieval, citations, safe tool use.
Safety & ethics	Guardrails, PII handling, ACLs, audit logs.
Observability & cost	Metrics, dashboards, budgets, routing.
Evaluation	Golden set, metrics (groundedness, relevance), CI gate.
UX / Demo	Shippable polish; real user feedback captured.
Portfolio readiness	Public GitHub repo, README, demo video/screenshots.
Impact narrative	Quantified time saved, $ saved, or CSAT lift.

Pass: 30/50 minimum; 40/50 for distinction. **Demo day:** 10 min presentation + 5 min live demo + 5 min Q&A; 3 reviewers per project.

Congratulations.

You are now a Claude AI Engineer.

Next steps

- Publish your five capstones on GitHub with polished READMEs.
- Record a demo-day video for each capstone.
- Apply for AI Engineer, LLM Engineer, or AI Architect roles.
- Contribute back — open source a prompt, a tool, or a dataset.